科技公司高频算法面试题解题思路 ## 目录 1. [数组与字符串](#数组与字符串) 2. [双指针](#双指针) 3. [滑动窗口](#滑动窗口) 4. [链表](#链表) 5. [栈与队列](#栈与队列) 6. [二叉树](#二叉树) 7. [回溯算法](#回溯算法) 8. [动态规划](#动态规划) 9. [图算法](#图算法) 10. [排序与搜索](#排序与搜索) --- ## 数组与字符串 ### 1. 两数之和 (LeetCode 1) **题目链接**: [LeetCode 1 – Two Sum](https://leetcode.com/problems/two-sum/) | [力扣 1 – 两数之和](https://leetcode.cn/problems/two-sum/) **难度**: 简单 **思路**: 使用哈希表存储已遍历的数字及其索引，一次遍历即可找到结果。 ```python def twoSum(nums, target): """ 时间复杂度: O(n) 空间复杂度: O(n) """ hashmap = {} for i, num in enumerate(nums): complement = target – num if complement in hashmap: return [hashmap[complement], i] hashmap[num] = i return [] ``` ### 2. 最长子串无重复字符 (LeetCode 3) **题目链接**: [LeetCode 3 – Longest Substring Without Repeating Characters](https://leetcode.com/problems/longest-substring-without-repeating-characters/) | [力扣 3 – 无重复字符的最长子串](https://leetcode.cn/problems/longest-substring-without-repeating-characters/) **难度**: 中等 **思路**: 滑动窗口 + 哈希集合，维护一个不重复字符的窗口。 ```python def lengthOfLongestSubstring(s): """ 时间复杂度: O(n) 空间复杂度: O(min(n, m)) m为字符集大小 """ char_set = set() left = 0 max_length = 0 for right in range(len(s)): while s[right] in char_set: char_set.remove(s[left]) left += 1 char_set.add(s[right]) max_length = max(max_length, right – left + 1) return max_length ``` **算法详解**: **核心思想**: 滑动窗口维护一个无重复字符的子串 – `left`和`right`是窗口的左右边界 – `char_set`存储窗口内的字符，用于O(1)检测重复 – `max_length`记录历史最大窗口长度（只增不减） **逐步演示** (s = "abcabcbb"): ``` 步骤1: right=0, 窗口=[a], max_length=1 步骤2: right=1, 窗口=[a,b], max_length=2 步骤3: right=2, 窗口=[a,b,c], max_length=3 步骤4: right=3, s[3]='a'重复 while循环移除s[0]='a' 窗口=[b,c,a], max_length=3 步骤5: right=4, s[4]='b'重复 while循环移除s[1]='b' 窗口=[c,a,b], max_length=3 步骤6–8: 窗口继续收缩，但max_length保持为3 ``` **关键点**: 1. **为什么用while不是if?** 重复字符可能在窗口任何位置，不只是最左侧 ```python # 示例: s="abba", right=2时 # 重复的'b'在索引1（不是最左侧） # while需要执行2次才能移除它 ``` 2. **max_length的作用**: 保存历史最大值，即使后续窗口缩小也不会减小 ```python max_length = max(max_length, right – left + 1) # ↑ ↑ # 历史最大 当前窗口长度 ``` 3. **时间复杂度为何是O(n)?** 虽然有嵌套循环，但每个字符最多访问2次（right加入1次，left移除1次） ### 3. 最长回文子串 (LeetCode 5) **题目链接**: [LeetCode 5 – Longest Palindromic Substring](https://leetcode.com/problems/longest-palindromic-substring/) | [力扣 5 – 最长回文子串](https://leetcode.cn/problems/longest-palindromic-substring/) **难度**: 中等 **思路**: 中心扩展法，从每个字符向两边扩展寻找回文串。 ```python def longestPalindrome(s): """ 时间复杂度: O(n^2) 空间复杂度: O(1) """ def expand_around_center(left, right): while left >= 0 and right < len(s) and s[left] == s[right]: left –= 1 right += 1 return right – left – 1 start = 0 max_len = 0 for i in range(len(s)): len1 = expand_around_center(i, i) # 奇数长度 len2 = expand_around_center(i, i + 1) # 偶数长度 current_max = max(len1, len2) if current_max >

max_len: max_len = current_max start = i - (current_max - 1) // 2 return s[start:start + max_len] ``` --- ## 双指针 ### 4. 三数之和 (LeetCode 15) **题目链接**: [LeetCode 15 - 3Sum](https://leetcode.com/problems/3sum/) | [力扣 15 - 三数之和](https://leetcode.cn/problems/3sum/) **难度**: 中等 **思路**: 排序 + 双指针，固定一个数，用双指针寻找另外两个数。 ```python def threeSum(nums): """ 时间复杂度: O(n²) 空间复杂度: O(1) """ nums.sort() result = [] for i in range(len(nums) - 2): if i > 0 and nums[i] == nums[i - 1]: continue left, right = i + 1, len(nums) - 1 while left < right: total = nums[i] + nums[left] + nums[right] if total < 0: left += 1 elif total > 0: right -= 1 else: result.append([nums[i], nums[left], nums[right]]) while left < right and nums[left] == nums[left + 1]: left += 1 while left < right and nums[right] == nums[right - 1]: right -= 1 left += 1 right -= 1 return result ``` ### 5. 盛最多水的容器 (LeetCode 11) **题目链接**: [LeetCode 11 - Container With Most Water](https://leetcode.com/problems/container-with-most-water/) | [力扣 11 - 盛最多水的容器](https://leetcode.cn/problems/container-with-most-water/) **难度**: 中等 **思路**: 双指针从两端向中间移动，每次移动较小的一边。 ```python def maxArea(height): """ 时间复杂度: O(n) 空间复杂度: O(1) """ left, right = 0, len(height) - 1 max_area = 0 while left < right: width = right - left current_area = width * min(height[left], height[right]) max_area = max(max_area, current_area) if height[left] < height[right]: left += 1 else: right -= 1 return max_area ``` --- ## 滑动窗口 ### 6. 最小覆盖子串 (LeetCode 76) **题目链接**: [LeetCode 76 - Minimum Window Substring](https://leetcode.com/problems/minimum-window-substring/) | [力扣 76 - 最小覆盖子串](https://leetcode.cn/problems/minimum-window-substring/) **难度**: 困难 **思路**: 滑动窗口 + 哈希表，维护窗口内字符计数。 ```python from collections import Counter def minWindow(s, t): """ 时间复杂度: O(m + n) 空间复杂度: O(m + n) """ if not s or not t: return "" dict_t = Counter(t) required = len(dict_t) left, right = 0, 0 formed = 0 window_counts = {} ans = float("inf"), None, None while right < len(s): char = s[right] window_counts[char] = window_counts.get(char, 0) + 1 if char in dict_t and window_counts[char] == dict_t[char]: formed += 1 while left <= right and formed == required: char = s[left] if right - left + 1 < ans[0]: ans = (right - left + 1, left, right) window_counts[char] -= 1 if char in dict_t and window_counts[char] < dict_t[char]: formed -= 1 left += 1 right += 1 return "" if ans[0] == float("inf") else s[ans[1]:ans[2] + 1] ``` --- ## 链表 ### 7. 反转链表 (LeetCode 206) **题目链接**: [LeetCode 206 - Reverse Linked List](https://leetcode.com/problems/reverse-linked-list/) | [力扣 206 - 反转链表](https://leetcode.cn/problems/reverse-linked-list/) **难度**: 简单 **思路**: 迭代或递归，改变指针方向。 ```python class ListNode: def __init__(self, val=0, next=None): self.val = val self.next = next def reverseList(head): """ 时间复杂度: O(n) 空间复杂度: O(1) """ prev = None current = head while current: next_temp = current.next current.next = prev prev = current current = next_temp return prev ``` ### 8. 合并两个有序链表 (LeetCode 21) **题目链接**: [LeetCode 21 - Merge Two Sorted Lists](https://leetcode.com/problems/merge-two-sorted-lists/) | [力扣 21 - 合并两个有序链表]

(https://leetcode.cn/problems/merge-two-sorted-lists/) **难度**: 简单 **思路**: 双指针比较，构建新链表。 ```python def mergeTwoLists(l1, l2): """ 时间复杂度: O(m + n) 空间复杂度: O(1) """ dummy = ListNode(0) current = dummy while l1 and l2: if l1.val < l2.val: current.next = l1 l1 = l1.next else: current.next = l2 l2 = l2.next current = current.next current.next = l1 if l1 else l2 return dummy.next ``` ### 9. 环形链表 II (LeetCode 142) **题目链接**: [LeetCode 142 - Linked List Cycle II](https://leetcode.com/problems/linked-list-cycle-ii/) | [力扣 142 - 环形链表 II](https://leetcode.cn/problems/linked-list-cycle-ii/) **难度**: 中等 **思路**: 快慢指针检测环，数学推导找到入口。 ```python def detectCycle(head): """ 时间复杂度: O(n) 空间复杂度: O(1) """ if not head or not head.next: return None slow = fast = head # 检测是否有环 while fast and fast.next: slow = slow.next fast = fast.next.next if slow == fast: break if not fast or not fast.next: return None # 找到环的入口 slow = head while slow != fast: slow = slow.next fast = fast.next return slow ``` --- ## 栈与队列 ### 10. 有效的括号 (LeetCode 20) **题目链接**: [LeetCode 20 - Valid Parentheses](https://leetcode.com/problems/valid-parentheses/) | [力扣 20 - 有效的括号](https://leetcode.cn/problems/valid-parentheses/) **难度**: 简单 **思路**: 使用栈匹配括号。 ```python def isValid(s): """ 时间复杂度: O(n) 空间复杂度: O(n) """ stack = [] mapping = {')': '(', '}': '{', ']': '['} for char in s: if char in mapping: top_element = stack.pop() if stack else '#' if mapping[char] != top_element: return False else: stack.append(char) return not stack ``` ### 11. 最小栈 (LeetCode 155) **题目链接**: [LeetCode 155 - Min Stack](https://leetcode.com/problems/min-stack/) | [力扣 155 - 最小栈](https://leetcode.cn/problems/min-stack/) **难度**: 中等 **思路**: 使用辅助栈存储最小值。 ```python class MinStack: """ 每个操作时间复杂度: O(1) 空间复杂度: O(n) """ def __init__(self): self.stack = [] self.min_stack = [] def push(self, val): self.stack.append(val) if not self.min_stack or val <= self.min_stack[-1]: self.min_stack.append(val) def pop(self): if self.stack: if self.stack[-1] == self.min_stack[-1]: self.min_stack.pop() self.stack.pop() def top(self): return self.stack[-1] if self.stack else None def getMin(self): return self.min_stack[-1] if self.min_stack else None ``` --- ## 二叉树 ### 12. 二叉树的最大深度 (LeetCode 104) **题目链接**: [LeetCode 104 - Maximum Depth of Binary Tree](https://leetcode.com/problems/maximum-depth-of-binary-tree/) | [力扣 104 - 二叉树的最大深度](https://leetcode.cn/problems/maximum-depth-of-binary-tree/) **难度**: 简单 **思路**: 递归或层序遍历。 ```python class TreeNode: def __init__(self, val=0, left=None, right=None): self.val = val self.left = left self.right = right def maxDepth(root): """ 时间复杂度: O(n) 空间复杂度: O(h) h为树高 """ if not root: return 0 return 1 + max(maxDepth(root.left), maxDepth(root.right)) ``` ### 13. 二叉树的层序遍历 (LeetCode 102) **题目链接**: [LeetCode 102 - Binary Tree Level Order Traversal](https://leetcode.com/problems/binary-tree-level-order-traversal/) | [力扣 102 - 二叉树的层序遍历](https://leetcode.cn/problems/binary-tree-level-order-traversal/) **难度**: 中等 **思路**: BFS使用队列。 ```python from collections import deque def levelOrder(root): """ 时间复杂度: O(n) 空间复杂度: O(n) """ if not root: return [] result = [] queue = deque([root]) while queue: level_size = len(queue) current_level

= [] for _ in range(level_size): node = queue.popleft() current_level.append(node.val) if node.left: queue.append(node.left) if node.right: queue.append(node.right) result.append(current_level) return result ``` ### 14. 二叉树的最近公共祖先 (LeetCode 236) **题目链接**: [LeetCode 236 – Lowest Common Ancestor of a Binary Tree](https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/) | [力扣 236 – 二叉树的最近公共祖先](https://leetcode.cn/problems/lowest-common-ancestor-of-a-binary-tree/) **难度**: 中等 **思路**: 递归查找，后序遍历。 ```python def lowestCommonAncestor(root, p, q): """ 时间复杂度: O(n) 空间复杂度: O(h) """ if not root or root == p or root == q: return root left = lowestCommonAncestor(root.left, p, q) right = lowestCommonAncestor(root.right, p, q) if left and right: return root return left if left else right ``` --- ## 回溯算法 ### 15. 全排列 (LeetCode 46) **题目链接**: [LeetCode 46 – Permutations](https://leetcode.com/problems/permutations/) | [力扣 46 – 全排列](https://leetcode.cn/problems/permutations/) **难度**: 中等 **思路**: 回溯法，使用visited数组标记。 ```python def permute(nums): """ 时间复杂度: O(n!) 空间复杂度: O(n) """ result = [] def backtrack(path): if len(path) == len(nums): result.append(path[:]) return for num in nums: if num not in path: path.append(num) backtrack(path) path.pop() backtrack([]) return result ``` ### 16. 子集 (LeetCode 78) **题目链接**: [LeetCode 78 – Subsets](https://leetcode.com/problems/subsets/) | [力扣 78 – 子集](https://leetcode.cn/problems/subsets/) **难度**: 中等 **思路**: 回溯法，每个元素选或不选。 ```python def subsets(nums): """ 时间复杂度: O(2^n) 空间复杂度: O(n) """ result = [] def backtrack(start, path): result.append(path[:]) for i in range(start, len(nums)): path.append(nums[i]) backtrack(i + 1, path) path.pop() backtrack(0, []) return result ``` ### 17. 单词搜索 (LeetCode 79) **题目链接**: [LeetCode 79 – Word Search](https://leetcode.com/problems/word-search/) | [力扣 79 – 单词搜索](https://leetcode.cn/problems/word-search/) **难度**: 中等 **思路**: DFS + 回溯，标记访问过的格子。 ```python def exist(board, word): """ 时间复杂度: O(m*n*4^L) L为word长度 空间复杂度: O(L) """ if not board: return False rows, cols = len(board), len(board[0]) def dfs(r, c, index): if index == len(word): return True if (r < 0 or r >= rows or c < 0 or c >= cols or board[r][c] != word[index]): return False temp = board[r][c] board[r][c] = '#' found = (dfs(r+1, c, index+1) or dfs(r-1, c, index+1) or dfs(r, c+1, index+1) or dfs(r, c-1, index+1)) board[r][c] = temp return found for i in range(rows): for j in range(cols): if dfs(i, j, 0): return True return False ``` --- ## 动态规划 ### 18. 爬楼梯 (LeetCode 70) **题目链接**: [LeetCode 70 – Climbing Stairs](https://leetcode.com/problems/climbing-stairs/) | [力扣 70 – 爬楼梯](https://leetcode.cn/problems/climbing-stairs/) **难度**: 简单 **思路**: 斐波那契数列，dp[i] = dp[i-1] + dp[i-2]。 ```python def climbStairs(n): """ 时间复杂度: O(n) 空间复杂度: O(1) """ if n <= 2: return n prev2, prev1 = 1, 2 for i in range(3, n + 1): current = prev1 + prev2 prev2 = prev1 prev1 = current return prev1 ``` ### 19. 最大子数组和 (LeetCode 53) **题目链接**: [LeetCode 53 – Maximum Subarray](https://leetcode.com/problems/maximum-

subarray/) | [力扣 53 – 最大子数组和](https://leetcode.cn/problems/maximum-subarray/) **难度**: 中等 **思路**: Kadane算法，动态规划。 ```python def maxSubArray(nums): """ 时间复杂度: O(n) 空间复杂度: O(1) """ max_sum = current_sum = nums[0] for num in nums[1:]: current_sum = max(num, current_sum + num) max_sum = max(max_sum, current_sum) return max_sum ``` ### 20. 零钱兑换 (LeetCode 322) **题目链接**: [LeetCode 322 – Coin Change](https://leetcode.com/problems/coin-change/) | [力扣 322 – 零钱兑换](https://leetcode.cn/problems/coin-change/) **难度**: 中等 **思路**: 完全背包问题，dp[i]表示凑成金额i的最少硬币数。 ```python def coinChange(coins, amount): """ 时间复杂度: O(amount * len(coins)) 空间复杂度: O(amount) """ dp = [float('inf')] * (amount + 1) dp[0] = 0 for i in range(1, amount + 1): for coin in coins: if i >= coin: dp[i] = min(dp[i], dp[i – coin] + 1) return dp[amount] if dp[amount] != float('inf') else –1 ``` ### 21. 最长递增子序列 (LeetCode 300) **题目链接**: [LeetCode 300 – Longest Increasing Subsequence](https://leetcode.com/problems/longest-increasing-subsequence/) | [力扣 300 – 最长递增子序列](https://leetcode.cn/problems/longest-increasing-subsequence/) **难度**: 中等 **思路**: dp[i]表示以nums[i]结尾的最长递增子序列长度。 ```python def lengthOfLIS(nums): """ 时间复杂度: O(n²) 或 O(n log n) 使用二分查找 空间复杂度: O(n) """ if not nums: return 0 dp = [1] * len(nums) for i in range(1, len(nums)): for j in range(i): if nums[i] > nums[j]: dp[i] = max(dp[i], dp[j] + 1) return max(dp) ``` --- ## 图算法 ### 22. 岛屿数量 (LeetCode 200) **题目链接**: [LeetCode 200 – Number of Islands](https://leetcode.com/problems/number-of-islands/) | [力扣 200 – 岛屿数量](https://leetcode.cn/problems/number-of-islands/) **难度**: 中等 **思路**: DFS或BFS遍历网格，标记访问过的陆地。 ```python def numIslands(grid): """ 时间复杂度: O(m*n) 空间复杂度: O(m*n) """ if not grid: return 0 rows, cols = len(grid), len(grid[0]) count = 0 def dfs(r, c): if (r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == '0'): return grid[r][c] = '0' dfs(r+1, c) dfs(r–1, c) dfs(r, c+1) dfs(r, c–1) for i in range(rows): for j in range(cols): if grid[i][j] == '1': count += 1 dfs(i, j) return count ``` ### 23. 课程表 (LeetCode 207) **题目链接**: [LeetCode 207 – Course Schedule](https://leetcode.com/problems/course-schedule/) | [力扣 207 – 课程表](https://leetcode.cn/problems/course-schedule/) **难度**: 中等 **思路**: 拓扑排序检测有向图是否有环。 ```python from collections import defaultdict, deque def canFinish(numCourses, prerequisites): """ 时间复杂度: O(V + E) 空间复杂度: O(V + E) """ graph = defaultdict(list) in_degree = [0] * numCourses for course, prereq in prerequisites: graph[prereq].append(course) in_degree[course] += 1 queue = deque([i for i in range(numCourses) if in_degree[i] == 0]) count = 0 while queue: course = queue.popleft() count += 1 for next_course in graph[course]: in_degree[next_course] –= 1 if in_degree[next_course] == 0: queue.append(next_course) return count == numCourses ``` --- ## 排序与搜索 ### 24. 二分查找 (LeetCode 704) **题目链接**: [LeetCode 704 – Binary Search](https://leetcode.com/problems/binary-search/) | [力扣 704 – 二分查找](https://leetcode.cn/problems/binary-search/) **难度**: 简单 **思路**: 标准二分查找模板。

```python def search(nums, target): """ 时间复杂度: O(log n) 空间复杂度: O(1) """ left, right = 0, len(nums) – 1 while left <= right: mid = left + (right – left) // 2 if nums[mid] == target: return mid elif nums[mid] < target: left = mid + 1 else: right = mid – 1 return –1 ``` ### 25. 搜索旋转排序数组 (LeetCode 33) **题目链接**: [LeetCode 33 – Search in Rotated Sorted Array](https://leetcode.com/problems/search-in-rotated-sorted-array/) | [力扣 33 – 搜索旋转排序数组](https://leetcode.cn/problems/search-in-rotated-sorted-array/) **难度**: 中等 **思路**: 修改的二分查找，判断哪一半是有序的。 ```python def searchRotated(nums, target): """ 时间复杂度: O(log n) 空间复杂度: O(1) """ left, right = 0, len(nums) – 1 while left <= right: mid = left + (right – left) // 2 if nums[mid] == target: return mid # 左半部分有序 if nums[left] <= nums[mid]: if nums[left] <= target < nums[mid]: right = mid – 1 else: left = mid + 1 # 右半部分有序 else: if nums[mid] < target <= nums[right]: left = mid + 1 else: right = mid – 1 return –1 ``` ### 26. 前K个高频元素 (LeetCode 347) **题目链接**: [LeetCode 347 – Top K Frequent Elements](https://leetcode.com/problems/top-k-frequent-elements/) | [力扣 347 – 前 K 个高频元素](https://leetcode.cn/problems/top-k-frequent-elements/) **难度**: 中等 **思路**: 哈希表计数 + 堆排序或桶排序。 ```python from collections import Counter import heapq def topKFrequent(nums, k): """ 时间复杂度: O(n log k) 空间复杂度: O(n) """ count = Counter(nums) return heapq.nlargest(k, count.keys(), key=count.get) ``` --- ## 高频题型总结 ### 数据结构选择 – **哈希表**: 快速查找、去重、计数 – **栈**: 括号匹配、单调栈、表达式求值 – **队列**: BFS、滑动窗口 – **堆**: TopK问题、中位数 – **双端队列**: 滑动窗口最大值 ### 算法模板 – **双指针**: 有序数组、链表、字符串 – **滑动窗口**: 子串、子数组问题 – **二分查找**: 有序数组、搜索空间 – **DFS**: 树、图遍历、回溯 – **BFS**: 最短路径、层序遍历 – **动态规划**: 最优子结构、重叠子问题 ### 时间复杂度目标 – **O(1)**: 哈希表查找 – **O(log n)**: 二分查找、平衡树 – **O(n)**: 单次遍历、双指针 – **O(n log n)**: 排序、堆操作 – **O(n^2)**: 双重循环（优化目标） ### 面试技巧 1. **理解题意**: 确认输入输出、边界条件 2. **举例分析**: 用具体例子梳理思路 3. **暴力解法**: 先想到可行解 4. **优化方案**: 分析瓶颈，选择合适数据结构 5. **代码实现**: 注意边界、空值处理 6. **测试验证**: 普通案例、边界案例 --- **准备建议**: – 每天刷2-3道题，坚持3个月 – 按类型刷题，总结模板 – 重点掌握高频题（前150题） – 注重代码质量和时间复杂度分析 – 模拟面试练习口头表达 **参考资源**: – [LeetCode 热门 100 题](https://leetcode.cn/problemset/hot-100/) – [LeetCode 精选 TOP 面试题](https://leetcode.cn/problemset/top-interview-questions/) – 《剑指Offer》 – 《程序员代码面试指南》
```